

SQL DML Practice Exercise

Database Setup

Run the following script first. **Do not modify this portion.**

```
DROP DATABASE IF EXISTS StoreDB;
CREATE DATABASE StoreDB;
USE StoreDB;

CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL
);

CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(8,2),
    Quantity INT,
    CategoryID INT,

    FOREIGN KEY (CategoryID)
        REFERENCES Category(CategoryID)
);

INSERT INTO Category VALUES
    (1, 'Electronics'),
    (2, 'Office Supplies'),
    (3, 'Furniture');

INSERT INTO Product VALUES
    (101, 'Wireless Mouse', 24.99, 20, 1),
    (102, 'Keyboard', 39.99, 15, 1),
    (103, 'Notebook', 4.99, 100, 2),
    (104, 'Desk Chair', 149.99, 8, 3);
```

Exercise 1

Write an INSERT statement to add the following product:

```
ProductID: 105
ProductName: USB Cable
Price: 12.99
Quantity: 30
CategoryID: 1
```

Exercise 2

Insert the following product, but **explicitly specify the column names** in your `INSERT` statement:

```
ProductID: 106
ProductName: Stapler
Price: 8.50
Quantity: 25
CategoryID: 2
```

Exercise 3

Insert the following product while listing the columns in this order:

```
ProductName
ProductID
CategoryID
Quantity
Price
```

The product information is:

```
ProductID: 107
ProductName: Office Desk
Price: 299.99
Quantity: 5
CategoryID: 3
```

The order of the values must correspond to the order of the columns you specify.

Exercise 4

Use **one INSERT statement** to add all three of these products:

ProductID	ProductName	Price	Quantity	CategoryID
108	Monitor	199.99	10	1
109	Printer Paper	7.99	50	2
110	Bookshelf	89.99	6	3

Exercise 5

Insert a product with the following information, but **do not provide a value for Price**:

```
ProductID: 111
ProductName: Pencil Set
Quantity: 40
CategoryID: 2
```

After inserting it, determine what value MySQL stores in the `Price` column.

Exercise 6 — Predict the Result

What will happen if you execute:

```
INSERT INTO Product
VALUES (101, 'Webcam', 49.99, 10, 1);
```

Explain **why** before running the statement.

Exercise 7 — Predict the Result

What will happen if you execute:

```
INSERT INTO Product
VALUES (112, NULL, 25.00, 10, 1);
```

Explain **why** before running the statement.

Exercise 8

Change the price of the `Wireless Mouse` with `ProductID = 101` to:

29.99

Exercise 9

Change both the price and quantity of `ProductID = 102`:

Price: 44.99
Quantity: 20

Use **one UPDATE statement**.

Exercise 10

Increase the quantity of every product in Category 2 to:

75

Your statement should intentionally update **multiple rows**.

Exercise 11

Change every product currently priced at:

4.99

to:

5.49

Exercise 12 — Think Before Executing

Consider:

```
UPDATE Product  
SET Quantity = 0;
```

Answer these questions **before executing it**:

1. How many rows will this statement affect?
 2. Why?
 3. What is missing from the statement?
-

Exercise 13

Delete the product whose:

`ProductID = 104`

Use the primary key in the `WHERE` clause.

Exercise 14

Delete the product named:

`Notebook`

using `ProductName` in the `WHERE` clause.

Then answer:

Why is deleting by `ProductID` normally safer than deleting by `ProductName`?

Exercise 15

Delete **all** products belonging to **Category 2**.

You should intentionally delete multiple rows using:

`CategoryID = 2`

Exercise 16

Assume there are still products with:

`CategoryID = 1`

What do you expect to happen if you execute:

```
DELETE FROM Category
WHERE CategoryID = 1;
```

Run it and record the MySQL error.

Then explain why MySQL prevents the deletion.

Exercise 17

After finishing the other exercises, execute:

```
DELETE FROM Product;
```

Then determine:

1. Does the `Product` table still exist?
2. Do its columns still exist?
3. Are there any rows remaining?